Loop

What is Loop?

A loop helps in executing one or more statements up to a desired number of times.

1

For Loop

Iterates over a sequence such as a list or a tuple.

2

While Loop

Executes a block of code repeatedly as long as a specified condition remains true.

For Loop

What is For Loop?

01 · CONCEPTS

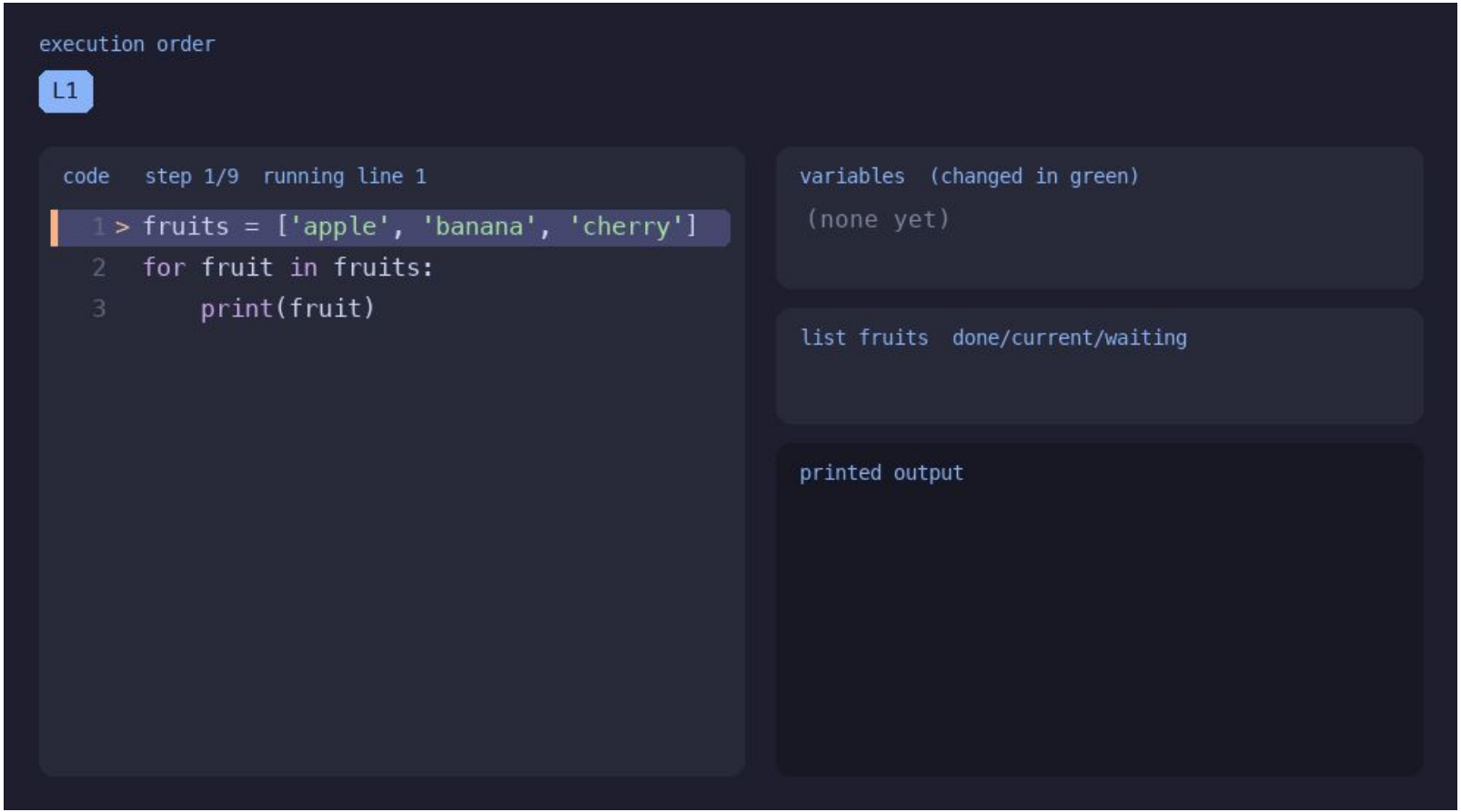
```
# SYNTAX
for item in iterable:
    # code block
    print(item)

# EXAMPLE
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)
```

The **'for'** loop is used to iterate over a sequence (list, tuple, string) or other iterable objects.

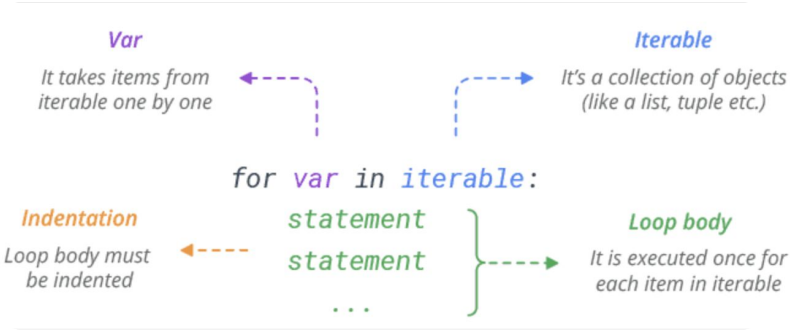
Insight: the loop variable (item) takes on each value in the sequence, one at a time.

Visualize Code Execution



For Loop

SYNTAX



Insight: the loop variable (var) takes on each value in the sequence, one at a time.

Code visualizer:

- https://programiz.pro/code-visualizer/python
- https://pythontutor.com/visualize.html#mode=display
- https://learn-code-with-vengleab.vercel.app/

Code Example

```
[1]: fruits = ['apple', 'banana', 'cherry']
     for fruit in fruits:
         print(fruit)

     apple
     banana
     cherry
```

For Loop with List or Tuple

```
# For Loop
fruits = ['apple', 'banana', 'orange']
for fruit in fruits:
    print("I want to " + fruit + "!!!")
```

```
I want to apple!!!
I want to banana!!!
I want to orange!!!
```

For loop with dictionary

```
person = {'name': 'John', 'age': 25, 'gender': 'Male'}

for key in person:
    print(f"Value of key: {key} ===> {person[key]}")
```

```
Value of key: name ===> John
Value of key: age ===> 25
Value of key: gender ===> Male
```

Using Range(n) with For Loop

- range(n) is commonly used with for loop to generates a sequence of numbers starting from **0** up to **n - 1**

Note: Range change be multiple syntax
- range(n)
- range(start, stop)
- range(start, stop, step)

```
for x in range(5):
    print("Value of " + str(x))
```

```
Value of 0
Value of 1
Value of 2
Value of 3
Value of 4
```

While Loop

What is While Loop?

01 · CONCEPTS

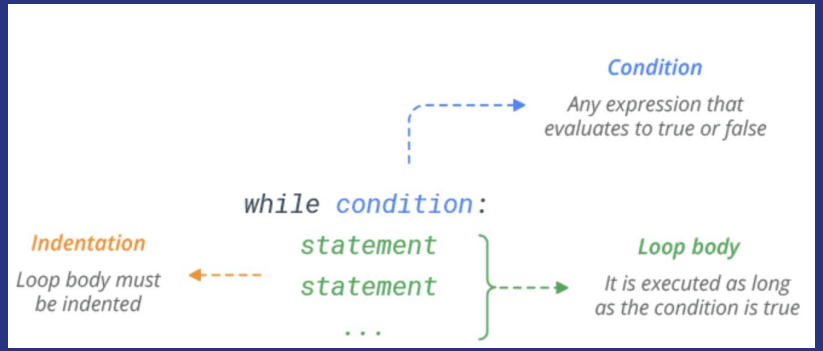
Executes a block of code repeatedly as long as a specified condition remains true.

EXECUTION FLOW

1. Evaluate condition
2. True → run body
3. Go back to step 1
4. False → exit loop

Insight: always ensure your condition eventually becomes False, or use break — otherwise it runs forever!

Syntax



Example

```
1  count = 0
2  while count < 5:
3      count = count + 1
4      print(count)
```

Visualize Code Execution

execution order

L1

code step 1/18 running line 1

```
1 > count = 0
2   while count < 5:
3       count = count + 1
4       print(count)
5
```

variables (changed in green)

(none yet)

printed output

Run until correct

[10]:
```
ready = False
while not ready:
    user_input = input("Type 'yes' to continue: ")
    if user_input.lower() == "yes":
        ready = True

print("Continuing...")
```

```
Type 'yes' to continue:  No
Type 'yes' to continue:  Y
Type 'yes' to continue:  N
Type 'yes' to continue:  Yes
Continuing...
```

Run multiple times

Similar to range(10) with for loop

```
step = 0
while step <10:
    step += 1 # can be written as: step = step + 1
    print(f"Execution times: {step}")
```

```
Execution times: 1
Execution times: 2
Execution times: 3
Execution times: 4
Execution times: 5
Execution times: 6
Execution times: 7
Execution times: 8
Execution times: 9
Execution times: 10
```

Transfer Statements

JUMP AROUND

Transfer Statements

Keywords that modify the normal top-to-bottom flow by jumping execution to a different point — think of them as traffic signs.

break

Exit the loop immediately.

```
for i in range(10):
    if i == 5:
        break
    print(i)  # 0–4 only
```

```
0
1
2
3
4
```

continue

Skip the current iteration.

```
for i in range(5):
    if i == 2:
        continue
    print(i)  # skip 2
```

```
0
1
3
4
```

pass

Placeholder — do nothing.

```
for i in range(5):
    pass  # TODO later
```

More with Loop

Nested Loops

A loop inside a loop

- The inner loop runs to completion for every single pass of the outer loop.
- Useful for grids, pairs, and coordinates — anywhere you need i and j together.
- range(3) twice gives 9 total iterations — 3 outer × 3 inner.

```
for i in range(3):
    for j in range(3):
        print(i, j)
```

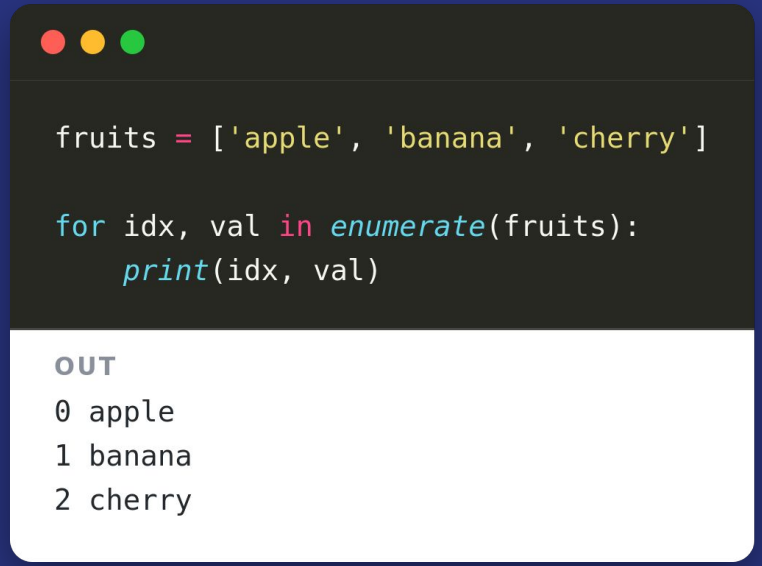
OUT

```
0 0
0 1
0 2
1 0
1 1
1 2
2 0
2 1
2 2
```

enumerate()

Index and value, together

- enumerate() wraps a list and yields (index, value) pairs.
- No need to track a counter by hand or call len().
- Cleaner and more Pythonic than looping over range(len(list)).



- **Insight:** use enumerate() instead of range(len(list)) for cleaner, more Pythonic code.

zip()

Pair up parallel lists

- zip() walks two (or more) lists side by side.
- Each step returns one item from each list, in the same order.
- Stops automatically at the shortest list if lengths differ.

```
names = ['Alice', 'Bob']
scores = [95, 87]

for n, s in zip(names, scores):
    print(n, s)
```

OUT

```
Alice 95
Bob 87
```

List Comprehensions

Syntax: `[expression for item in iterable if condition]`

Basic — Squares

```
# Before
squares = []
for i in range(5):
    squares.append(i**2)

# After
squares = [i**2 for i in range(5)]
# [0, 1, 4, 9, 16]
```

With Filter — Evens

```
# Before
evens = []
for i in range(10):
    if i % 2 == 0:
        evens.append(i)

# After
evens = [i for i in range(10) if i % 2 == 0]
# [0, 2, 4, 6, 8]
```

Dict Comprehension

```
# Before
squares = {}
for i in range(4):
    squares[i] = i**2

# After
squares = {i: i**2 for i in range(4)}
# {0:0, 1:1, 2:4, 3:9}
```

Practices: Foundations

Part 1: Loop with For Loop

Counting, stepping, and summing with range()

Part 1: Loop with For Loop

1 Count from 1 to 10

- Print the numbers from 1 to 10.
  - Hint: use range(1, 11)

Expected output:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

2 Print Even Numbers

- Print every even number from 2 to 20 (inclusive).
  - Hint: use a step of 2 in range() or using %

Expected output:

2, 4, 6, 8, 10, 12, 14, 16, 18, 20

3 Sum of Numbers

- Calculate and print the sum of all numbers from 1 to 100.

Expected output:

Sum: 5050

4 Loop Through a List

- Given the list: ['apple', 'banana', 'cherry']
- print each fruit on its own line.

Expected output:

apple
banana
cherry
date

Part 2: Loops with While

Countdowns, tables, and searching through data

Part 2: Loops with While

5 Countdown

- Countdown from 10 to 1.
- Then print "Done!"

Expected output:

10, 9, 8, 7, 6, 5, 4, 3, 2, 1
Done!

6 Multiplication Table

- Print the 5 times table — from 5 x 1 up to 5 x 10.

Expected output:

5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50

7 Count Vowels in a String

- Count how many vowels (a, e, i, o, u) appear in "programming".

Expected output:

Vowel count: 3

8 Find the Largest Number

- Find the largest value in [14, 58, 2, 99, 43].
  - Hint: Game
  - Constraint: don't use max()

Expected output:

Largest number: 99

Practices: More with loop

Practices: More with loop

1 Transaction Categorization

- Loop through the account amounts
- if/else: positive -> Income, else -> Expense
- Print each amount with its label

```
amounts = [150.0, -45.5, -12.0, 3000.0, -8.99]

# TODO: for each amount, print Income or Expense
for amt in amounts:
    pass
```

Expected output:

$150.00 -> Income
-$45.50 -> Expense
-$12.00 -> Expense
$3000.00 -> Income
-$8.99 -> Expense

2 High-Value Transaction Filter

- Loop through daily transaction values
- if value > $1,000: add it to the total
- Print the final flagged total

```
values = [25, 499, 1200, 85, 5000, 310]
total = 0

# TODO: if value > 1000, add to total
for v in values:
    pass
```

Expected output:

Total high-value transactions: $6200

Practices: More with loop

3 Credit Score Classifier

- if score >= 700 -> Prime
- elif 620-699 -> Near-Prime
- else -> Subprime

```
scores = [720, 580, 810, 640, 520]

# TODO: if/elif/else classify each score
for score in scores:
    pass
```

Expected output:

720 -> Prime
580 -> Subprime
810 -> Prime
640 -> Near-Prime
520 -> Subprime

4 Skip Zero-Fee Accounts

- if fee == 0.0: continue to skip it
- Print only the active fee charges
- Practice the continue keyword

```
fees = [12.0, 0.0, 15.0, 0.0, 8.5]

# TODO: continue past any $0.0 fee
for fee in fees:
    pass
```

Expected output:

Fee: $12.00
Fee: $15.00
Fee: $8.50

Practices: More with loop

5

```
prices = [100, 102, 101, 88, 82, 79]

# TODO: if price < 90, print + break
for price in prices:
    pass
```

Your task

- Print each trade price as you go
- if price < $90: print Trading Halted
- break immediately — stop processing

Expected output:

100
102
101